

Princess Sumaya University for Technology
جامعة الأميرة سميّة للتكنولوجيا

Cloud Computing & Security

25541

Course Project Proposal

CloudTrap: A Multi-Cloud Honeypot Deception Network

Student Name Katrin Zagha **Student No.** 20210858
Student Name Tala Ghosheh **Student No.** 20220123

Instructor: Farah Abudabaseh

April 12, 2026

1. Problem Statement

Modern cloud infrastructures are under constant attack. Organizations deploying services on public cloud platforms face threats including unauthorized access attempts, credential stuffing, API abuse, and data exfiltration. While existing security tools focus on detecting and blocking known threats, they are largely reactive, they respond after an attacker has already made contact with real assets.

A significant gap exists in the current security landscape: most organizations have no mechanism to study attacker behavior in a controlled environment. They do not know what specific attack methods are being used against their cloud infrastructure, which credentials are being tested, or what reconnaissance techniques attackers employ before launching a real intrusion.

Furthermore, organizations that rely on a single cloud provider have limited visibility into cross-cloud threat patterns. Attackers often probe multiple cloud platforms simultaneously, and there is currently no lightweight, open-source solution that provides unified deception-based detection across multiple cloud service providers (CSPs).

CloudTrap addresses this gap by deploying a coordinated network of realistic fake cloud services "honeypots" across AWS and GCP. These decoy services attract and log attacker activity without exposing any real infrastructure, enabling continuous passive intelligence gathering about active threat actors targeting cloud environments.

2. Cloud Services to be Used

CloudTrap makes use of services across two cloud providers: AWS and GCP to satisfy the multi-cloud requirement and demonstrate cross-platform integration.

2.1 Amazon Web Services (AWS)

- **AWS EC2** hosts the Dockerized honeypot services (fake S3-compatible endpoint and fake REST API). EC2 was chosen for its flexibility in configuring open ports and network interfaces that realistically mimic exposed cloud services.
- **Amazon CloudWatch** collects and stores all interaction logs generated by the honeypot services. CloudWatch Logs streams are used to capture attacker requests in real time, and CloudWatch alarms notify the system when interaction thresholds are exceeded.
- **AWS IAM** enforces least-privilege access control between all system components. IAM roles are scoped tightly so that the honeypot EC2 instance can only write to CloudWatch and has no access to any real data or production resources.
- **AWS Secrets Manager** stores all API keys and credentials securely, ensuring no sensitive values are hardcoded in source code or Docker images.
- **Amazon ECR (Elastic Container Registry)** stores Docker images built by the CI/CD pipeline. Only signed, pipeline-validated images are pulled for deployment.

2.2 Google Cloud Platform (GCP)

- **GCP Compute Engine** hosts a third honeypot service: a fake login portal mimicking a cloud management console. Deploying on a separate CSP increases the realism of the deception network and captures threats targeting GCP-specific infrastructure.
- **GCP Cloud Logging** aggregates logs from the GCP-hosted honeypot and forwards them to the centralized analyzer service, enabling unified visibility across both cloud environments.
- **GCP Secret Manager** manages GCP-side credentials following the same least-privilege principle as AWS Secrets Manager.
- **GCP Artifact Registry** stores Docker images for GCP-deployed containers, mirroring the ECR setup on the AWS side.

3. Architecture Overview and Data Flow

The system is composed of four logical layers: the deception layer, the logging layer, the analysis layer, and the presentation layer. All services are containerized using Docker and managed via Docker Compose. Figure 1 below provides a visual overview of the full architecture.

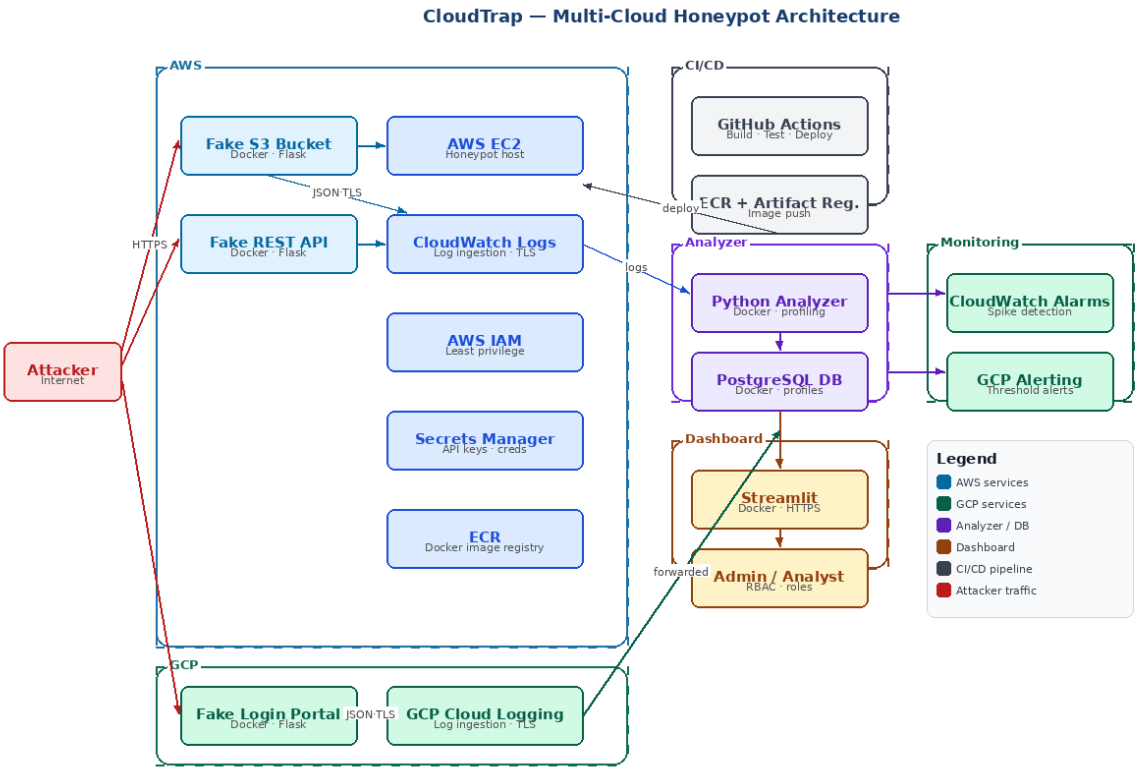


Figure 1: CloudTrap multi-cloud honeypot architecture: AWS (blue), GCP (green), Analyzer (purple), Dashboard (amber), CI/CD (gray)

3.1 End-to-End Data Flow

The following sequence describes how data moves through the system from the moment an attacker interacts with a honeypot to the moment it appears on the dashboard:

- **Step 1:** Attacker initiates an HTTP/HTTPS request to a publicly exposed honeypot endpoint (Fake S3, Fake API, or Fake Login Portal).
- **Step 2:** The honeypot service (Python Flask, Docker container) captures the full request: source IP, HTTP method, headers, body payload, and timestamp. It returns a realistic but empty response to avoid detection.
- **Step 3:** The honeypot serializes the interaction as a structured JSON log entry and writes it to its cloud logging service (CloudWatch Logs on AWS, Cloud Logging on GCP) over TLS.
- **Step 4:** A log forwarder service normalizes GCP logs into the same JSON schema used by AWS and ships them to a shared storage bucket accessible by the analyzer.
- **Step 5:** The Python analyzer (Docker) polls for new log entries, performs deduplication, IP geolocation enrichment, and pattern detection (credential stuffing, port scanning, API enumeration).
- **Step 6:** Attacker profiles are written to a PostgreSQL database (containerized). PostgreSQL was chosen over SQLite because it supports concurrent reads from the dashboard while the analyzer is writing SQLite's file-locking model would cause bottlenecks under moderate load.
- **Step 7:** The Streamlit dashboard reads from PostgreSQL and presents a real-time feed, a world map of attack origins, metric charts, and ranked attacker profiles.

3.2 Deception Layer

CloudTrap deploys four honeypot services across both cloud providers, using a combination of low-interaction and high-interaction designs. Low-interaction honeypots simulate only the surface of a service and are safe and always-on; high-interaction honeypots present a near-real environment to capture deeper attacker behavior such as post-login activity and injection attempts.

- **Fake S3 Bucket Service (AWS EC2, Docker)** → low-interaction. A Python Flask application mimicking the AWS S3 REST API. Any request to list, get, or upload objects is silently logged without returning real data.
- **Fake REST API (AWS EC2, Docker)** → low-interaction. A generic API endpoint simulating a business application backend. Accepts and logs any incoming requests, capturing headers, payloads, and source IPs.
- **Fake Login Portal (GCP Compute Engine, Docker)** → low-interaction. A web page styled to resemble a cloud management console. Submitted credentials are logged and the attacker receives a generic error.
- **Dummy Web App + PostgreSQL (AWS EC2, extra-isolated Docker network)** → high-interaction. A realistic web application backed by a PostgreSQL instance with fake user records, deployed with zero outbound network routing to ensure complete containment even if an attacker gains a foothold.

3.3 Logging Layer

Each honeypot writes structured JSON logs to its respective cloud logging service over TLS. The JSON schema includes: timestamp, source IP, HTTP method, endpoint path, request headers, payload (sanitized), and cloud provider tag. The GCP log forwarder normalizes these logs and ships them to a shared location accessible to the analyzer.

3.4 Analysis Layer

The Python analyzer consumes logs from both cloud environments and performs: deduplication and schema normalization across CSPs, IP geolocation enrichment using a public GeoIP API, pattern detection for credential stuffing, scanning behavior, and API enumeration, and attacker session profiling grouped by source IP with a full activity timeline. An optional rule-based attack classification module labels attacker behavior into categories such as credential theft, reconnaissance, or data exfiltration attempts providing an innovation layer beyond the core requirements.

3.5 Presentation Layer : Dashboard and RBAC

The Streamlit dashboard displays a live interaction feed, a geographic attack origin map, attacker profile cards, and metric charts showing request rate and attack frequency over time. Access to the dashboard is role-based:

- Admin role → full access including raw log export and system configuration.
- Analyst role → read-only access to interaction feeds and attacker profiles.

Role enforcement uses dashboard-level authentication with separate credentials per role, backed by Secrets Manager on both clouds.

3.6 Scalability Strategy

Honeypots can receive burst traffic during active attack campaigns. The following scalability measures are incorporated into the design:

- AWS EC2 Auto Scaling groups are configured for the honeypot instances, allowing horizontal scaling when CPU or network thresholds are exceeded.
- GCP Compute Engine managed instance groups provide equivalent auto-scaling on the GCP side.
- CloudWatch Logs and GCP Cloud Logging are fully managed services with no ingestion rate limits at the scale expected for this project.
- The analyzer is stateless by design, meaning multiple analyzer instances can run in parallel against the same log storage if throughput demands it.

3.7 Failure and Recovery Plan

The system is designed to degrade gracefully rather than lose data:

- If the analyzer crashes, Docker restart policies (restart: always) automatically restart the container. Logs remain safely stored in CloudWatch and GCP Cloud Logging until the analyzer recovers and reprocesses them.

- If the GCP log forwarder fails, logs continue to accumulate in GCP Cloud Logging. Upon recovery, the forwarder replays any unforwarded log entries using offset-based checkpointing.
- If the dashboard becomes unavailable, it has no effect on data collection the deception and logging layers operate independently.
- Backup storage: logs are also exported to an S3 bucket on a daily schedule as a secondary archive.

4. Security Considerations

Because CloudTrap is itself a security tool, its design must be carefully secured to prevent misuse or lateral movement from the honeypot environment into real infrastructure.

4.1 Isolation

All honeypot services are deployed in isolated VPCs with no peering or routing to any production network. Egress rules are strictly limited, honeypot containers can only write logs outbound; they cannot initiate connections to internal services. This prevents an attacker who discovers they are in a honeypot from pivoting to real infrastructure.

4.2 IAM and Least Privilege

AWS IAM roles are scoped to write-only access on CloudWatch Logs. No honeypot service holds credentials granting read access to any AWS resource. On GCP, service accounts follow the same principle. All credentials are stored in Secrets Manager never hardcoded in source code or Docker images.

4.3 Encryption and Secure Communication

All log data in transit is encrypted via TLS. Data at rest in CloudWatch and GCP Cloud Logging is encrypted using provider-managed keys by default. The dashboard is only accessible over HTTPS and requires authentication before displaying any data.

4.4 Threat Model

The following table identifies the key assets in the system, the threats against them, and the mitigations in place:

Asset	Threat	Mitigation
Log storage	Log poisoning: attacker submits malformed entries	Logs sanitized and escaped before storage; analyzer validates schema before processing

Analyzer service	Privilege escalation: attacker exploits analyzer	Isolated Docker container with no cloud credentials; read-only access to log storage only
Honeypot endpoints	DdoS: flood of requests exhausts EC2 resources	Auto Scaling absorbs burst traffic; CloudWatch alarms trigger on threshold breach
Dashboard	Unauthorized access to attacker intelligence	Role-based auth (Admin / Analyst); HTTPS only; credentials in Secrets Manager
Docker images	Malicious image substitution in CI/CD	Images built and scanned in GitHub Actions; only signed images deployed

4.5 Containment of Captured Data

Credentials and payloads submitted by attackers are treated as potentially malicious. They are stored in escaped, sanitized form only and are never executed or evaluated. Log storage is private and accessible only to the analyzer, with access logging enabled to detect unauthorized reads.

5. Testing Strategy

The following testing plan ensures the system functions correctly, handles high load, and resists security risks.

5.1 Functional Testing

Automated scripts will simulate attacker behavior by sending HTTP requests to each honeypot endpoint, including S3 list/get/put operations, REST API calls with various payloads, and fake login credential submissions. Tests will verify that all interactions are correctly captured, logged in the expected JSON schema, forwarded to the analyzer, and appear on the dashboard.

5.2 Security Testing

Security tests will verify that: no real data is returned by any honeypot under any request pattern, no credentials or payloads stored in logs can be executed or re-injected, the honeypot VPC has no route to any production resource, and the dashboard correctly enforces role-based access (Admin vs Analyst).

5.3 Performance Testing

A load test will simulate a high-volume attack scenario by sending 1,000 requests per minute to the honeypots over a sustained period. The test will verify that: log ingestion does not drop any events, auto-scaling triggers correctly under load, the analyzer processes the backlog within an acceptable time window, and the dashboard remains responsive throughout.

6. CI/CD Pipeline

A GitHub Actions pipeline automates building, testing, and deploying all Docker containers to both cloud providers. The pipeline stages are:

- **Build** Docker images for all services are built on every push to the main branch.
- **Secret Scanning** GitHub's built-in secret scanning and a Gitleaks step block any commit containing hardcoded credentials or API keys before the build proceeds.
- **Image Vulnerability Scanning** Trivy scans every built image for known CVEs. The pipeline fails if any critical vulnerabilities are found, preventing insecure images from being deployed.
- **Test** unit tests for the analyzer and functional tests for honeypot endpoints are executed automatically.
- **Push** validated and signed images are pushed to Amazon ECR (AWS services) and GCP Artifact Registry (GCP services).
- **Deploy** updated images are pulled and redeployed to EC2 and GCP Compute Engine via SSH using zero-downtime Docker Compose container replacement.

This pipeline ensures every code change is automatically validated, security-scanned, and deployed consistently across both cloud environments directly satisfying the CI/CD and security requirements in the grading rubric.

7. Cost Consideration

CloudTrap uses AWS EC2 rather than serverless functions (e.g., AWS Lambda) because honeypot services require persistent, long-running processes that maintain open TCP connections and simulate real server behavior, a model fundamentally incompatible with Lambda's stateless, short-lived execution model. EC2 also allows precise control over exposed network ports and interface configuration, which is essential for realistic deception. The cost trade-off is acceptable: EC2 t3.micro instances fall within the AWS Free Tier for the project's duration, whereas Lambda costs would accumulate unpredictably under sustained attacker traffic. Regarding multi-cloud costs, operating honeypots on both AWS and GCP introduces a dual billing surface, compute hours on GCP Compute Engine (e2-micro, also Free Tier eligible), outbound data transfer fees for cross-cloud log forwarding, and two separate secret management services. These costs are kept minimal by selecting the smallest Free Tier instance types on both providers and routing inter-cloud log traffic through lightweight JSON payloads; however, teams scaling beyond a proof-of-concept should account for egress fees that can grow non-linearly with attacker interaction volume.

References

[1] N. Provos and T. Holz, Virtual Honeypots: From Botnet Tracking to Intrusion Detection. Addison-Wesley Professional, 2007.

[2] L. Spitzner, Honeypots: Tracking Hackers. Addison-Wesley, 2002.

[3] Google Cloud, "Google Cloud Security Foundations Guide," Google Cloud Architecture Framework, 2024. [Online]. Available: https://cloud.google.com/architecture/security-foundations

[4] M. Nawrocki, M. Wählisch, T. C. Schmidt, C. Keil, and J. Schoenfelder, "A Survey on Honeypot Software and Data Analysis," arXiv:1608.06249, 2016.

[5] A. Vetterl and R. Clayton, “Honware: A Virtual Honeypot Framework for Capturing CPE and IoT Traffic,” in Proc. APWG Symp. on Electronic Crime Research (eCrime), 2019, pp. 1–13.

[6] R. Baykara and Z. Z. Gürel, “A Survey on Potential Applications of Honeypot Technology in Intrusion Detection Systems,” International Journal of Computer Networks and Applications, vol. 6, no. 1, pp. 25–36, 2019.

[7] MITRE, “ATT&CK for Cloud,” MITRE Corporation, 2024. [Online]. Available: https://attack.mitre.org/matrices/enterprise/cloud/

[8] S. T. King and P. M. Chen, “Backtracking Intrusions,” in Proc. 19th ACM Symposium on Operating Systems Principles (SOSP), 2003, pp. 223–236.

[9] CSA, “Top Threats to Cloud Computing: Pandemic Eleven,” Cloud Security Alliance, 2022. [Online]. Available: https://cloudsecurityalliance.org/

[10] D. Dittrich and E. Kenneally, “The Menlo Report: Ethical Principles Guiding Information and Communication Technology Research,” U.S. Department of Homeland Security, 2012.

[11] T. Ristenpart, E. Tromer, H. Shacham, and S. Savage, “Hey, You, Get Off of My Cloud: Exploring Information Leakage in Third-Party Compute Clouds,” in Proc. ACM CCS, 2009, pp. 199–212.

[12] R. Rosen, Linux Kernel Networking: Implementation and Theory. Apress, 2014. (for Docker networking and VPC isolation concepts)